

ARTIFICIAL INTELLIGENCE

ITCS 6150/8150 – Spring 2025

Programming Project - 1

Solving the 8-puzzle problem using A* search algorithm

SUBMITTED BY :

SREEJA EDLA

sedla@charlotte.edu

Id: 801419758

Uma Maheshwar Eidigi

ueidigi@charlotte.edu

Id: 801454913

PROBLEM STATEMENT

The 8-puzzle problem is a puzzle invented by *Noyes Palmer Chapman*. It is played on a 3-by-3 grid with 8 square blocks labeled 1 through 8 and a blank square. The goal is to rearrange the blocks so that they are in order. We are permitted to slide blocks horizontally or vertically into the blank square. The following shows a sequence of legal moves from an initial board position to the goal position.

1	3			1		3		1	2	3		1	2	3		1	2	3
4	2	5	=>	4	2	5	=>	4		5	=>	4	5		=>	4	5	6
7	8	6		7	8	6		7	8	6		7	8	6		7	8	
initial												goal						

THE PROJECT

You are to implement A* search algorithm and apply it to the 8-puzzle problem, using a programming language of your choice. In addition to coding of A* search algorithm, provide problem formulation, operators, g-cost and two heuristic functions (h-cost) of the 8-puzzle problem. Your program should accept initial and goal states from user and will compute the best path.

Sample initial and goal states:

Note that, if it can't find a solution for a sample, simply mention no solution found. Also, '0' means blank square or vice versa.

Initial state:	Goal State:
1 2 3	1 2 3
7 4 5	8 6 4
6 8 0	7 5 0

Initial state:	Goal State:
2 8 1	3 2 1
3 4 6	8 0 4
7 5 0	7 5 6

Initial state:	Goal State:
7 2 4	1 2 3
5 6	4 5 6
8 3 1	7 8

INTRODUCTION

3.1 The 8-Puzzle

The **8-puzzle** consists of a 3×3 grid containing tiles labeled 1–8 and one blank (denoted **0**). A legal move slides one adjacent tile **up, down, left, or right** into the blank.

Objective: starting from an initial arrangement, reach the **goal arrangement**.

We model the problem as a search task:

$$\mathcal{P} = \langle S, A, T, c, s_0, s_g \rangle$$

where S is the set of board states, A is the set of actions {Up, Down, Left, Right}, T is the deterministic transition function, c is the step cost, s_0 is the initial state, and s_g is the goal state.

3.2 Definition

The 8-puzzle is a small sliding-tile game played on a 3×3 board with tiles numbered 1–8 and one blank space. A move slides a neighboring tile into the blank; the goal is to reach a target arrangement from a given starting arrangement. Despite its tiny size, the 8-puzzle is a classic testbed for search algorithms in Artificial Intelligence because it forces us to explore a state space efficiently and reason about “how close” a position is to the goal.

In this project we solve the 8-puzzle using the A^* search algorithm. At a high level, A^* always expands the next position that looks most promising based on two things: (1) how many moves we’ve already made to reach it, and (2) an estimate of how many moves remain. This balance helps A^* avoid wasting time wandering aimlessly while still guaranteeing the shortest solution when the estimate (called a heuristic) is well-behaved.

We implement and compare two simple, widely used heuristics for the 8-puzzle:

- **Misplaced tiles:** counts how many tiles are not currently in their goal spots.
- **Manhattan distance:** for each tile, counts how many horizontal and vertical steps it would take to slide that tile to its goal spot, then adds those up.

Intuitively, the first heuristic answers “how many tiles are wrong,” while the second answers “how far are the tiles from where they belong.” In practice, Manhattan distance usually gives A^* better guidance because it considers

s *how wrong* each tile is, not just *whether* it's wrong.

Beyond finding solutions, this project also focuses on **analysis**. For eight test cases, we run A* with both heuristics and report:

- the **solution path** (the sequence of moves),
- the **number of nodes generated** (how many states we created overall),
- the **number of nodes expanded** (how many states we actually pulled off the priority queue to explore).

We also include a short **summary table** to compare the two heuristics across the eight cases. Before searching, our program performs a quick **solvability check** so we don't waste time on impossible inputs (for the 3×3 puzzle, exactly half of all scrambled boards are unsolvable).

From an implementation perspective, the solver keeps a **priority queue** (the “open list”) of frontier states ordered by how promising they look, plus a **visited/best-cost map** (the “closed set”) to avoid revisiting weaker versions of the same board. Each move has equal cost (one step), and ties in the priority queue are broken consistently to keep results deterministic. The program accepts an initial and goal state from the user, runs A*, and prints the solution path and statistics. Clear inline comments and structured functions make the code easy to read and test.

Deliverables include: a short report (problem formulation, heuristics, algorithm design, experiments, and conclusions), the source code with comments, and a quick “how to run” note. The report highlights what we observed—for example, where Manhattan distance reduces the number of expanded nodes compared to misplaced tiles—and briefly discusses trade-offs and possible extensions (like enhanced heuristics or memory-frugal variants such as IDA*).

Overall, this project demonstrates how a well-chosen heuristic can turn a brute-force search into an effective problem solver, even on a deceptively simple game like the 8-puzzle.

3.2 Heuristic Functions

We use two standard heuristics for the 8-puzzle. In both, the blank (0) is ignored. Each heuristic returns a non-negative integer estimating how many moves remain to reach the goal. Both are **admissible** (never overestimate) and **consistent** (triangle inequality holds), so A* with either heuristic is optimal.

3.2.1 Misplaced Tiles (h_1)

Idea. Count how many tiles are not currently in their goal positions.

Why does it help. If a tile isn't in its goal spot, at least one move is needed to fix it.

- **Admissible:** A tile that's wrong needs ≥ 1 move; counting such tiles never overshoots the true distance
- **Consistent:** Moving one tile can change this count by at most 1, so $h(\text{current}) \leq 1 + h(\text{neighbor})$.

3.2.2 Manhattan Distance (h_2)

Idea. For each tile, count the number of horizontal + vertical steps needed to slide it to its goal cell; sum over all tiles.

Why does it help. In the 8-puzzle, tiles move in grid steps. The total “taxicab” distance is a natural lower bound on the number of moves needed.

- **Admissible:** Each tile must travel at least its Manhattan distance; summing lower bounds is still a lower bound.
- **Consistent:** Moving one step changes the sum by at most 1, so the triangle inequality holds.

A* ALGORITHM DESIGN

How A* solves the 8-puzzle (concept & choices)

- **Goal:** find the *shortest* move sequence from an initial board to the goal board, or report “No solution found.”
- **State & moves:** a state is a 3×3 board (0 = blank). Legal moves slide the blank **Up/Down/Left/Right** (no wrap).
- **Cost model:** each move costs **1** $\rightarrow$ path cost g = depth.
- **Heuristics (h):** we use **Misplaced Tiles** and **Manhattan Distance** (both ignore 0, both admissible & consistent).
- **Priority rule:** expand the node with **smallest $f=g+h$** next (A*).
Tie-break deterministically (e.g., prefer smaller h or use a FIFO counter).
- **Graph search:** keep a **best-g map** to avoid revisiting worse versions of the same board; this guarantees optimality with our heuristics.
- **Goal test:** check when a node is **popped** from the priority queue; the first time you pop the goal, the path is optimal.
- **Solvability check:** run inversion-parity test first; if unsolvable, exit early.
- **What you count for the report:**
 - **Nodes generated:** successors you **push** into the open list.
 - **Nodes expanded:** nodes you **pop** from the open list and actually **expand** (produce neighbors)

IMPLEMENTATION

We implement the 8-puzzle solver in **Python**, using **A*** search. The program prompts the user for both the **start** and **target** configurations.

Input

- Users enter the board as a **3×3 grid**, typing the nine values (use 0 for the blank) with **spaces** between entries.
- The user then chooses one of two **heuristics**:
 - **Misplaced Tiles**
 - **Manhattan Distance**

Output

After running A*, the program reports:

- The **solution path**: the ordered sequence of moves from the start to the goal.
- The **total nodes generated** during the search.
- The **total nodes expanded** (i.e., states removed from the frontier and explored).

Program Structure

The implementation is organized into the following key components:

- **Class PuzzleNode**
Encapsulates a puzzle configuration (state), path cost $g(n)$, heuristic cost $h(n)$, and provides methods to:
 - Get/set costs and predecessor nodes.
 - Compute total cost $f(n) = g(n) + h(n)$ for A*.
 - Generate neighbors by sliding tiles into the blank space.
- **Utility Functions**
 - `read_board_lines(prompt)`: Reads initial and goal states from the user (supports comma/space separated input).
 - `inversions(state), is_solvable(start, goal)`: Implements the inversion-parity check to ensure the input

is solvable before searching.

- *A Search Algorithm**

- Maintains a **priority queue (min-heap)** of states, ordered by $f(n)$.
- Uses a **g-score map** and **parent dictionary** to track the shortest discovered path to each state and reconstruct the solution path.
- Expands nodes by generating legal neighbor states (successors).
- Stops when the goal state is dequeued from the priority queue.

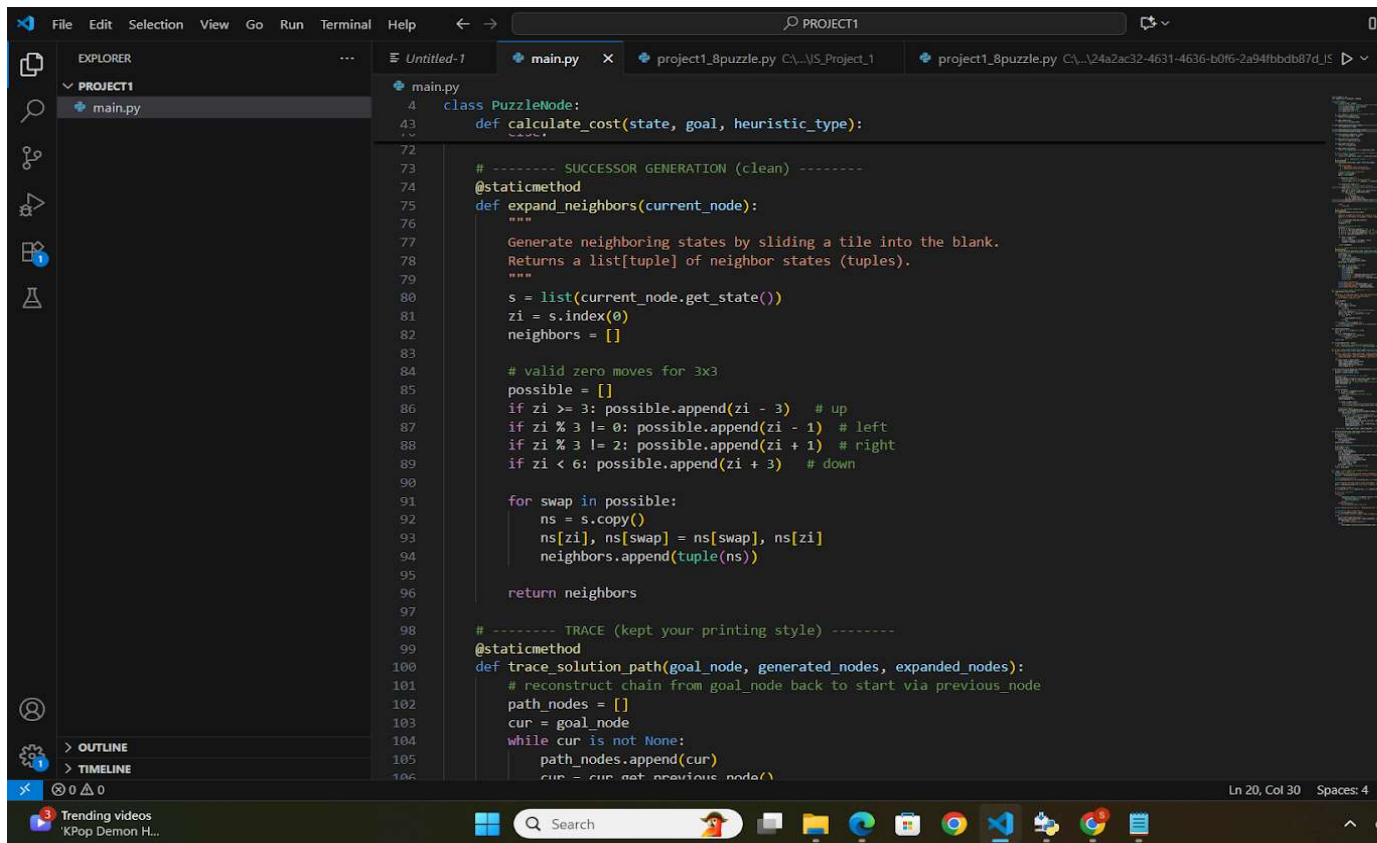
- **Solution Reconstruction**

- `build_solution_node_chain`: Reconstructs the path from goal to start using the parent mapping, computing correct $g(n)$ and $h(n)$ for each node.
- `PuzzleNode.trace_solution_path`: Pretty-prints the entire solution path, including intermediate boards, $g(n)$, $h(n)$, and $f(n)$ values.

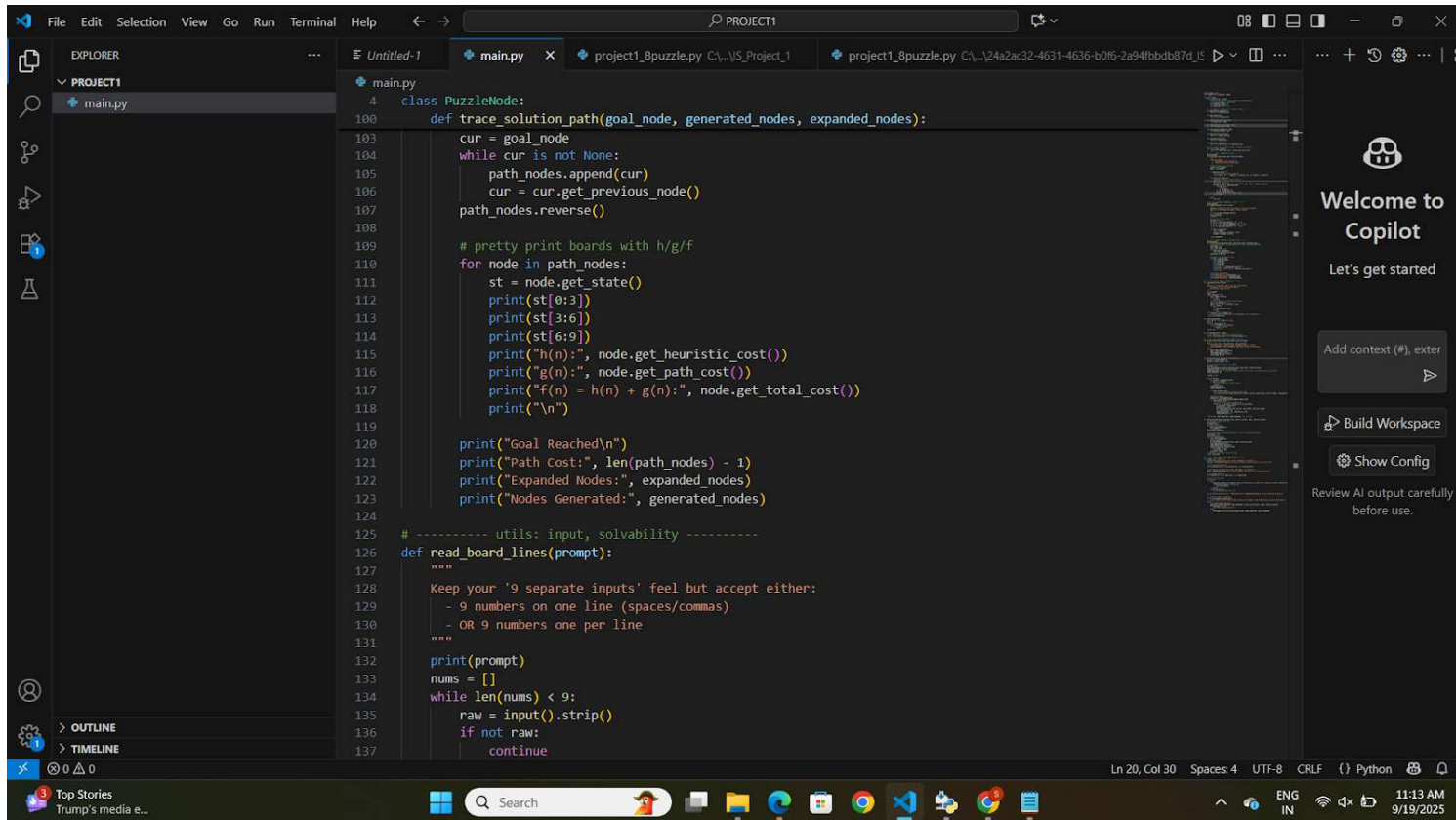
SOURCE CODE

```
1 import numpy as np
2 from heapq import heappush, heappop
3
4 class PuzzleNode:
5     def __init__(self, state):
6         # store as tuple for hashing; accept list/tuple/np array
7         self.current_state = tuple(state)
8         self.previous_node = None
9         self.heuristic_cost = 0
10        self.path_cost = 0 # g(n)
11
12        # ----- getters / setters (your original style) -----
13        def get_previous_node(self):
14            return self.previous_node
15
16        def get_state(self):
17            return self.current_state
18
19        def update_path_cost(self, cost):
20            self.path_cost = cost
21
22        def update_heuristic_cost(self, cost):
23            self.heuristic_cost = cost
24
25        def set_previous_node(self, node):
26            self.previous_node = node
27
28        def get_heuristic_cost(self):
29            return self.heuristic_cost
30
31        def get_path_cost(self):
32            return self.path_cost
33
34        def get_total_cost(self):
35            return self.path_cost + self.heuristic_cost
36
37        # allow nodes to be compared in heap if needed (tie-break)
```

```
38 def __lt__(self, other):
39     return self.get_total_cost() < other.get_total_cost()
40
41 # ----- COST / HEURISTICS (fixed) -----
42 @staticmethod
43 def calculate_cost(state, goal, heuristic_type):
44     """
45     heuristic type:
46     1 -> Misplaced tiles (ignoring 0)
47     2 -> Manhattan distance (ignoring 0)
48     """
49     # ensure tuples for consistency
50     state = tuple(state)
51     goal = tuple(goal)
52
53     if heuristic_type == 1:
54         # Misplaced tiles (ignore blank 0)
55         return sum(1 for i in range(9) if state[i] != 0 and state[i] != goal[i])
56
57     elif heuristic_type == 2:
58         # Manhattan distance (ignore blank 0)
59         cost = 0
60         # map goal positions
61         goal_pos = {tile: (idx // 3, idx % 3) for idx, tile in enumerate(goal)}
62         for idx, tile in enumerate(state):
63             if tile == 0:
64                 continue
65             r, c = divmod(idx, 3)
66             gr, gc = goal_pos[tile]
67             cost += abs(r - gr) + abs(c - gc)
68         return cost
69     else:
70         return 0
71
72 # ----- SUCCESSOR GENERATION (clean) -----
```



```
4 class PuzzleNode:
43     def calculate_cost(state, goal, heuristic_type):
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73     # ----- SUCCESSOR GENERATION (clean) -----
74     @staticmethod
75     def expand_neighbors(current_node):
76         """
77         Generate neighboring states by sliding a tile into the blank.
78         Returns a list[tuple] of neighbor states (tuples).
79         """
80         s = list(current_node.get_state())
81         zi = s.index(0)
82         neighbors = []
83
84         # valid zero moves for 3x3
85         possible = []
86         if zi >= 3: possible.append(zi - 3) # up
87         if zi % 3 != 0: possible.append(zi - 1) # left
88         if zi % 3 != 2: possible.append(zi + 1) # right
89         if zi < 6: possible.append(zi + 3) # down
90
91         for swap in possible:
92             ns = s.copy()
93             ns[zi], ns[swap] = ns[swap], ns[zi]
94             neighbors.append(tuple(ns))
95
96         return neighbors
97
98     # ----- TRACE (keep your printing style) -----
99     @staticmethod
100     def trace_solution_path(goal_node, generated_nodes, expanded_nodes):
101         # reconstruct chain from goal_node back to start via previous_node
102         path_nodes = []
103         cur = goal_node
104         while cur is not None:
105             path_nodes.append(cur)
106             cur = cur.get_previous_node()
```



```
100     def trace_solution_path(goal_node, generated_nodes, expanded_nodes):
103         cur = goal_node
104         while cur is not None:
105             path_nodes.append(cur)
106             cur = cur.get_previous_node()
107         path_nodes.reverse()
108
109         # pretty print boards with h/g/f
110         for node in path_nodes:
111             st = node.get_state()
112             print(st[0:3])
113             print(st[3:6])
114             print(st[6:9])
115             print("h(n):", node.get_heuristic_cost())
116             print("g(n):", node.get_path_cost())
117             print("f(n) = h(n) + g(n):", node.get_total_cost())
118             print("\n")
119
120         print("Goal Reached\n")
121         print("Path Cost:", len(path_nodes) - 1)
122         print("Expanded Nodes:", expanded_nodes)
123         print("Nodes Generated:", generated_nodes)
124
125     # ----- utils: input, solvability -----
126     def read_board_lines(prompt):
127         """
128         Keep your '9 separate inputs' feel but accept either:
129         - 9 numbers on one line (spaces/commas)
130         - OR 9 numbers one per line
131         """
132         print(prompt)
133         nums = []
134         while len(nums) < 9:
135             raw = input().strip()
136             if not raw:
137                 continue
```

This screenshot shows the first part of a Python script in a VS Code editor. The Explorer pane on the left shows a project named 'PROJECT1' with a file 'main.py'. The main editor displays the following code:

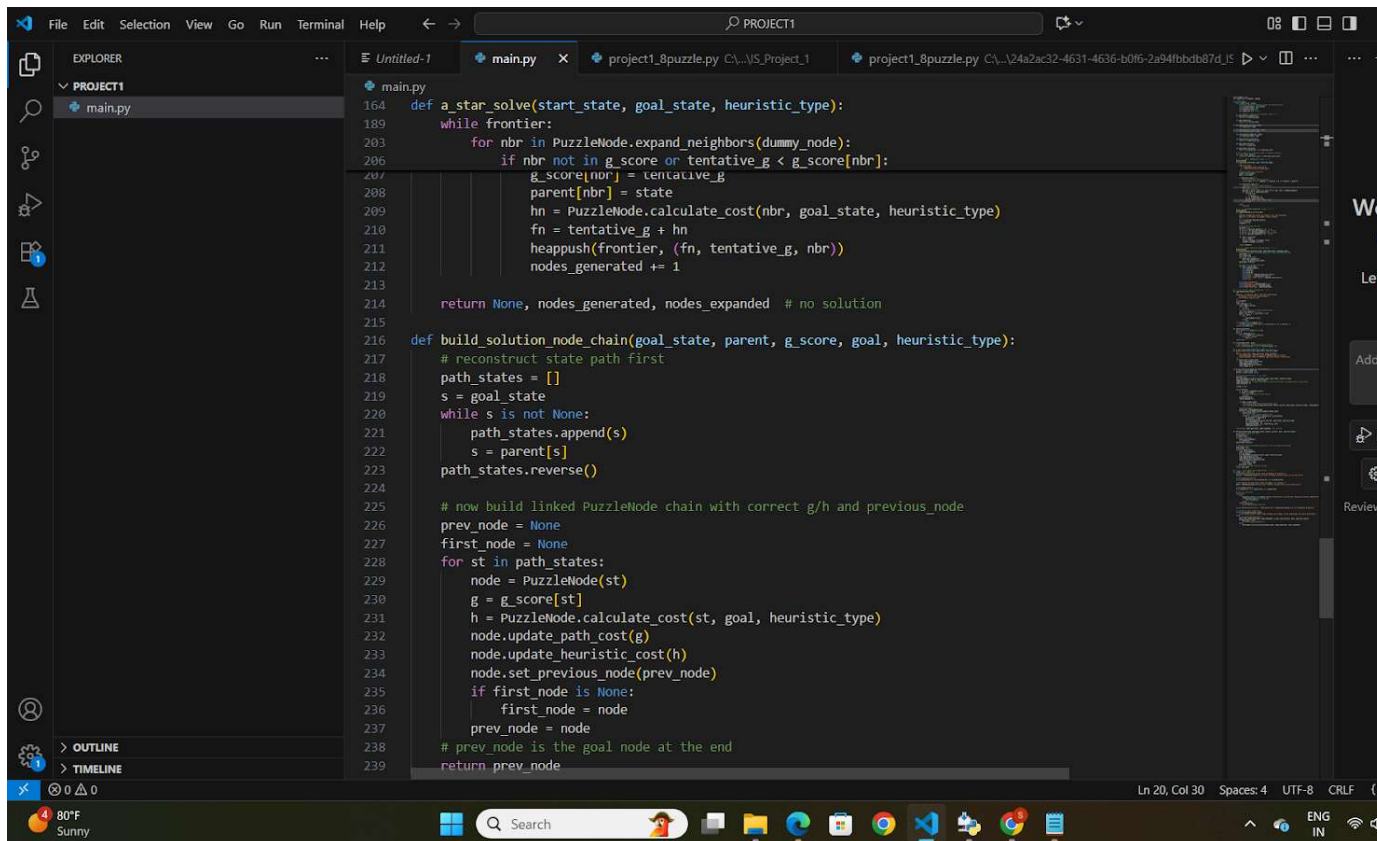
```
126 def read_board_lines(prompt):
134 while len(nums) < 9:
135     raw = input().strip()
136     if not raw:
137         continue
138     # allow a single line with multiple numbers
139     raw = raw.replace(" ", "")
140     parts = [p for p in raw.split() if p]
141     for p in parts:
142         try:
143             nums.append(int(p))
144         except:
145             pass
146     if set(nums) != set(range(0, 9)):
147         raise ValueError("State must be a permutation of 0..8 (0=blank).")
148     return tuple(nums[:9])
149
150 def inversions(state):
151     arr = [x for x in state if x != 0]
152     inv = 0
153     for i in range(len(arr)):
154         for j in range(i + 1, len(arr)):
155             if arr[i] > arr[j]:
156                 inv += 1
157     return inv
158
159 def is_solvable(start, goal):
160     # 3x3 solvability: inversion parity must match
161     return inversions(start) % 2 == inversions(goal) % 2
162
163 # ----- A* (correct counters & logic) -----
164 def a_star_solve(start_state, goal_state, heuristic_type):
165     """
166     Returns (goal_node, nodes_generated, nodes_expanded)
167     - nodes_generated: times we actually PUSHED a node to frontier
168     - nodes_expanded: times we POPPED a node from frontier (first time)
169     """
```

The status bar at the bottom indicates 'Ln 20, Col 30 Spaces: 4 UTF-8 CRLF'.

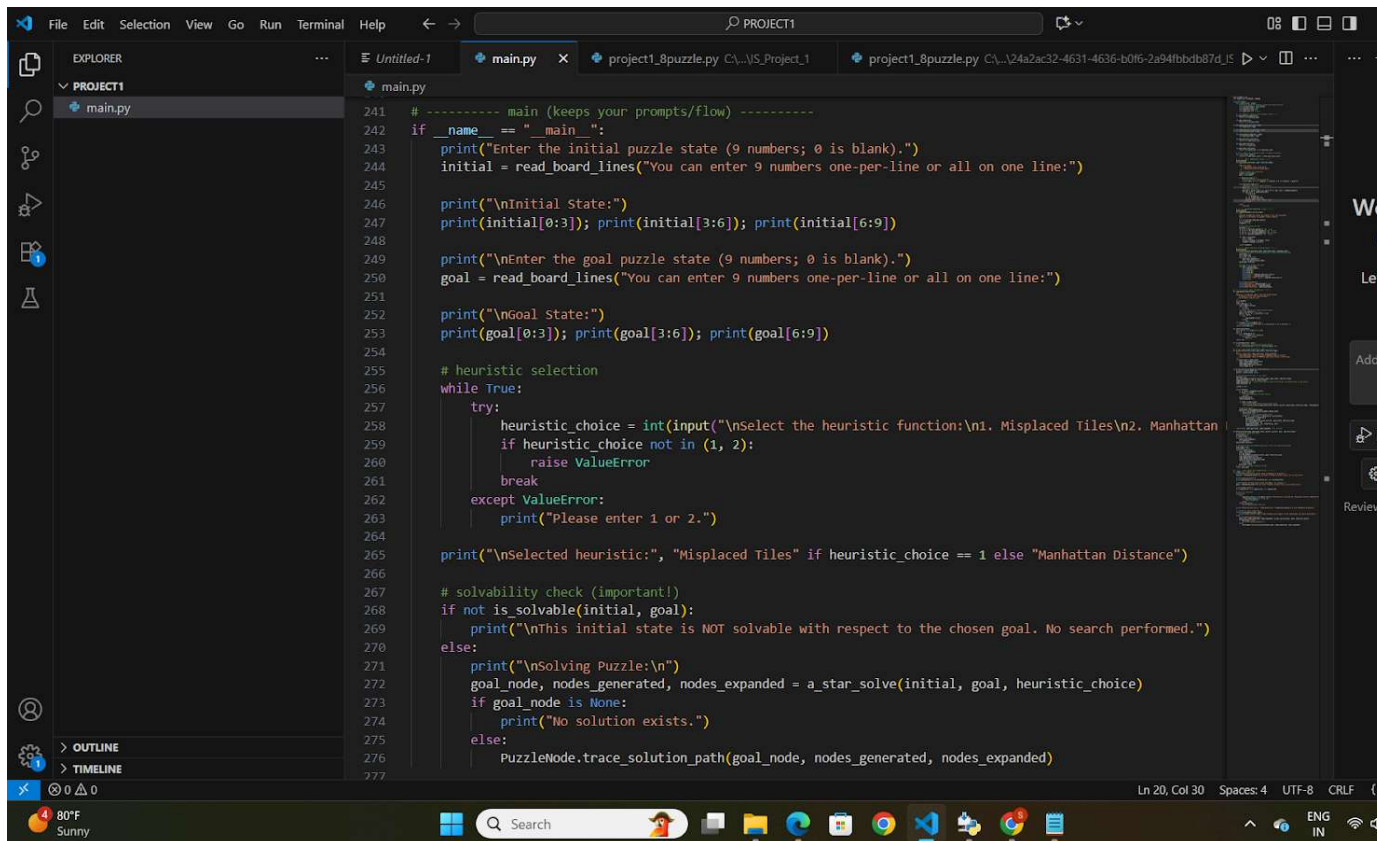
This screenshot shows the second part of the Python script in the VS Code editor. The main editor displays the following code:

```
164 def a_star_solve(start_state, goal_state, heuristic_type):
170     if start_state == goal_state:
171         node = PuzzleNode(start_state)
172         node.update_path_cost(0)
173         node.update_heuristic_cost(0)
174         return node, 0, 0
175
176     # g(n) best known cost and parent mapping
177     g_score = {start_state: 0}
178     parent = {start_state: None}
179
180     # priority queue entries: (f, g, state)
181     frontier = []
182     h0 = PuzzleNode.calculate_cost(start_state, goal_state, heuristic_type)
183     heappush(frontier, (h0, 0, start_state))
184     nodes_generated = 0 # only count when we push into the heap (excluding start if you prefer)
185     nodes_expanded = 0
186
187     closed = set()
188
189     while frontier:
190         f, g, state = heappop(frontier)
191         if state in closed:
192             # already expanded this best version
193             continue
194         closed.add(state)
195         nodes_expanded += 1
196
197         if state == goal_state:
198             # build PuzzleNode chain from parent/g_score
199             return build_solution_node_chain(state, parent, g_score, goal_state, heuristic_type), nodes_generated
200
201         # expand neighbors
202         dummy_node = PuzzleNode(state)
203         for nbr in PuzzleNode.expand_neighbors(dummy_node):
204             tentative_g = g + 1
205             # if never seen or found a better path
```

The status bar at the bottom indicates 'Ln 20, Col 30 Spaces: 4 UTF-8 CRLF'.



```
164 def a_star_solve(start_state, goal_state, heuristic_type):
189     while frontier:
203         for nbr in PuzzleNode.expand_neighbors(dummy_node):
206             if nbr not in g_score or tentative_g < g_score[nbr]:
207                 g_score[nbr] = tentative_g
208                 parent[nbr] = state
209                 hn = PuzzleNode.calculate_cost(nbr, goal_state, heuristic_type)
210                 fn = tentative_g + hn
211                 heappush(frontier, (fn, tentative_g, nbr))
212                 nodes_generated += 1
213
214     return None, nodes_generated, nodes_expanded # no solution
215
216 def build_solution_node_chain(goal_state, parent, g_score, goal, heuristic_type):
217     # reconstruct state path first
218     path_states = []
219     s = goal_state
220     while s is not None:
221         path_states.append(s)
222         s = parent[s]
223     path_states.reverse()
224
225     # now build linked PuzzleNode chain with correct g/h and previous_node
226     prev_node = None
227     first_node = None
228     for st in path_states:
229         node = PuzzleNode(st)
230         g = g_score[st]
231         h = PuzzleNode.calculate_cost(st, goal, heuristic_type)
232         node.update_path_cost(g)
233         node.update_heuristic_cost(h)
234         node.set_previous_node(prev_node)
235         if first_node is None:
236             first_node = node
237         prev_node = node
238     # prev_node is the goal node at the end
239     return prev_node
```



```
241 # ----- main (keeps your prompts/flow) -----
242 if __name__ == "__main__":
243     print("Enter the initial puzzle state (9 numbers; 0 is blank).")
244     initial = read_board_lines("You can enter 9 numbers one-per-line or all on one line:")
245
246     print("\nInitial State:")
247     print(initial[0:3]); print(initial[3:6]); print(initial[6:9])
248
249     print("\nEnter the goal puzzle state (9 numbers; 0 is blank).")
250     goal = read_board_lines("You can enter 9 numbers one-per-line or all on one line:")
251
252     print("\nGoal State:")
253     print(goal[0:3]); print(goal[3:6]); print(goal[6:9])
254
255     # heuristic selection
256     while True:
257         try:
258             heuristic_choice = int(input("\nSelect the heuristic function:\n1. Misplaced Tiles\n2. Manhattan Distance\n"))
259             if heuristic_choice not in (1, 2):
260                 raise ValueError
261             break
262         except ValueError:
263             print("Please enter 1 or 2.")
264
265     print("\nSelected heuristic:", "Misplaced Tiles" if heuristic_choice == 1 else "Manhattan Distance")
266
267     # solvability check (important!)
268     if not is_solvable(initial, goal):
269         print("\nThis initial state is NOT solvable with respect to the chosen goal. No search performed.")
270     else:
271         print("\nSolving Puzzle:\n")
272         goal_node, nodes_generated, nodes_expanded = a_star_solve(initial, goal, heuristic_choice)
273         if goal_node is None:
274             print("No solution exists.")
275         else:
276             PuzzleNode.trace_solution_path(goal_node, nodes_generated, nodes_expanded)
277
```


RESULTS

Example	Input	Manhattan Distance Heuristic Function	Misplaced Tiles Heuristic Function
Example 1	Initial State:	Total Nodes Generated = 18 Total Nodes Expanded = 10	Total Nodes Generated = 41 Total Nodes Expanded = 22
	1 2 3		
	7 4 5		
	6 8 0		
	Goal State:		
	1 2 3 8 6 4 7 5 0		
Example 2	Initial State:	Total Nodes Generated =12 Total Nodes Expanded = 7	Total Nodes Generated = 14 Total Nodes Expanded = 8
	2 8 1		
	3 4 6		
	7 5 0		
	Goal State:		
	3 2 1 8 0 4 7 5 6		

Example 3	Initial State: <table><tr><td>7</td><td>2</td><td>4</td></tr><tr><td>5</td><td>0</td><td>6</td></tr><tr><td>8</td><td>3</td><td>1</td></tr></table> Goal State: <table><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>6</td></tr><tr><td>7</td><td>8</td><td>0</td></tr></table>	7	2	4	5	0	6	8	3	1	1	2	3	4	5	6	7	8	0	Total Nodes Generated =451 Total Nodes Expanded = 283	Total Nodes Generated = 5750 Total Nodes Expanded = 3667
7	2	4																			
5	0	6																			
8	3	1																			
1	2	3																			
4	5	6																			
7	8	0																			
Example 4	Initial State: <table><tr><td>5</td><td>3</td><td>0</td></tr><tr><td>2</td><td>8</td><td>4</td></tr><tr><td>1</td><td>7</td><td>6</td></tr></table> Goal State: <table><tr><td>5</td><td>3</td><td>4</td></tr><tr><td>2</td><td>7</td><td>8</td></tr><tr><td>1</td><td>0</td><td>6</td></tr></table>	5	3	0	2	8	4	1	7	6	5	3	4	2	7	8	1	0	6	Total Nodes Generated = 7 Total Nodes Expanded = 4	Total Nodes Generated = 7 Total Nodes Expanded = 4
5	3	0																			
2	8	4																			
1	7	6																			
5	3	4																			
2	7	8																			
1	0	6																			
Example 5	Initial State: <table><tr><td>1</td><td>4</td><td>2</td></tr><tr><td>7</td><td>0</td><td>5</td></tr><tr><td>3</td><td>6</td><td>8</td></tr></table> Goal State:	1	4	2	7	0	5	3	6	8	Total Nodes Generated = 2753 Total Nodes Expanded = 1661	Total Nodes Generated = 15852 Total Nodes Expanded = 10332									
1	4	2																			
7	0	5																			
3	6	8																			

	<table><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>6</td></tr><tr><td>7</td><td>8</td><td>0</td></tr></table>	1	2	3	4	5	6	7	8	0											
1	2	3																			
4	5	6																			
7	8	0																			
Example 6	Initial State: <table><tr><td>5</td><td>1</td><td>3</td></tr><tr><td>4</td><td>2</td><td>6</td></tr><tr><td>7</td><td>8</td><td>0</td></tr></table> Goal State: <table><tr><td>5</td><td>1</td><td>3</td></tr><tr><td>4</td><td>2</td><td>0</td></tr><tr><td>7</td><td>8</td><td>6</td></tr></table>	5	1	3	4	2	6	7	8	0	5	1	3	4	2	0	7	8	6	Total Nodes Generated = 2 Total Nodes Expanded = 2	Total Nodes Generated = 2 Total Nodes Expanded = 2
5	1	3																			
4	2	6																			
7	8	0																			
5	1	3																			
4	2	0																			
7	8	6																			
Example 7	Initial State: <table><tr><td>2</td><td>8</td><td>3</td></tr><tr><td>1</td><td>0</td><td>4</td></tr><tr><td>7</td><td>6</td><td>5</td></tr></table> Goal State: <table><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>8</td><td>0</td><td>4</td></tr><tr><td>7</td><td>6</td><td>5</td></tr></table>	2	8	3	1	0	4	7	6	5	1	2	3	8	0	4	7	6	5	Total Nodes Generated = 9 Total Nodes Expanded = 5	Total Nodes Generated = 11 Total Nodes Expanded = 6
2	8	3																			
1	0	4																			
7	6	5																			
1	2	3																			
8	0	4																			
7	6	5																			
Example 8	Initial State: <table><tr><td>1</td><td>2</td><td>3</td></tr></table>	1	2	3	Total Nodes Generated = 63 Total Nodes Expanded = 37	Total Nodes Generated = 115 Total Nodes Expanded = 69															
1	2	3																			

	<table> <tr> <td>0</td><td>5</td><td>6</td></tr> <tr> <td>4</td><td>7</td><td>8</td></tr> </table> Goal State: <table> <tr> <td>1</td><td>2</td><td>3</td></tr> <tr> <td>4</td><td>5</td><td>0</td></tr> <tr> <td>6</td><td>7</td><td>8</td></tr> </table>	0	5	6	4	7	8	1	2	3	4	5	0	6	7	8	
0	5	6															
4	7	8															
1	2	3															
4	5	0															
6	7	8															

PROBLEM-SOLVING PATH

Input 1:

Initial State:

1	2	3
7	4	5
6	8	0

Goal State:

1	2	3
8	6	4
7	5	0

Manhattan Distance:

File Edit Selection View Go Run Terminal Help PROJECT1

EXPLORER PROJECT1 main.py

TERMINAL

Selected heuristic: Manhattan Distance

Solving Puzzle:

```
(1, 2, 3)
(7, 4, 5)
(6, 8, 0)
h(n): 8
g(n): 0
f(n) = h(n) + g(n): 8

(1, 2, 3)
(7, 4, 0)
(6, 8, 5)
h(n): 7
g(n): 1
f(n) = h(n) + g(n): 8

(1, 2, 3)
(7, 0, 4)
(6, 8, 5)
h(n): 6
g(n): 2
f(n) = h(n) + g(n): 8

(1, 2, 3)
(7, 8, 4)
(6, 0, 5)
h(n): 5
g(n): 3
f(n) = h(n) + g(n): 8
```

OUTLINE TIMELINE

Ln 20, Col 30 Spaces: 4 UTF-8 CRLF Python 11:26 AM 9/19/2025

File Edit Selection View Go Run Terminal Help PROJECT1

EXPLORER PROJECT1 main.py

TERMINAL

```
(1, 2, 3)
(7, 8, 4)
(0, 6, 5)
h(n): 4
g(n): 4
f(n) = h(n) + g(n): 8

(1, 2, 3)
(0, 8, 4)
(7, 6, 5)
h(n): 3
g(n): 5
f(n) = h(n) + g(n): 8

(1, 2, 3)
(8, 0, 4)
(7, 6, 5)
h(n): 2
g(n): 6
f(n) = h(n) + g(n): 8

(1, 2, 3)
(8, 6, 4)
(7, 0, 5)
h(n): 1
g(n): 7
f(n) = h(n) + g(n): 8

(1, 2, 3)
(8, 6, 4)
(7, 5, 0)
h(n): 0
g(n): 8
f(n) = h(n) + g(n): 8
```

OUTLINE TIMELINE

Ln 20, Col 30 Spaces: 4 UTF-8 CRLF Python 11:26 AM 9/19/2025

Misplaced Tiles Distance:

```
File Edit Selection View Go Run Terminal Help PROJECT1
EXPLORER
PROJECT1
main.py
TERMINAL
Selected heuristic: Misplaced Tiles
Solving Puzzle:
(1, 2, 3)
(7, 4, 5)
(6, 8, 0)
h(n): 5
g(n): 0
f(n) = h(n) + g(n): 5
(1, 2, 3)
(7, 4, 0)
(6, 8, 5)
h(n): 5
g(n): 1
f(n) = h(n) + g(n): 6
(1, 2, 3)
(7, 0, 4)
(6, 8, 5)
h(n): 4
g(n): 2
f(n) = h(n) + g(n): 6
(1, 2, 3)
(7, 8, 4)
(6, 0, 5)
h(n): 4
g(n): 3
f(n) = h(n) + g(n): 7
(1, 2, 3)
(7, 8, 4)
(0, 6, 5)
h(n): 4
g(n): 4
f(n) = h(n) + g(n): 8
Welcome to Copilot
Let's get started
Add context (#)
Build Workspace
Show Config
Review AI output carefully before use.
```

```
File Edit Selection View Go Run Terminal Help PROJECT1
EXPLORER
PROJECT1
main.py
TERMINAL
f(n) = h(n) + g(n): 8
(1, 2, 3)
(0, 8, 4)
(7, 6, 5)
h(n): 3
g(n): 5
f(n) = h(n) + g(n): 8
(1, 2, 3)
(8, 0, 4)
(7, 6, 5)
h(n): 2
g(n): 6
f(n) = h(n) + g(n): 8
(1, 2, 3)
(8, 6, 4)
(7, 0, 5)
h(n): 1
g(n): 7
f(n) = h(n) + g(n): 8
(1, 2, 3)
(8, 6, 4)
(7, 5, 0)
h(n): 0
g(n): 8
f(n) = h(n) + g(n): 8
Goal Reached
Path Cost: 8
Expanded Nodes: 22
Nodes Generated: 41
PS C:\Users\sree\Desktop\PROJECT1>
```

Input 2:

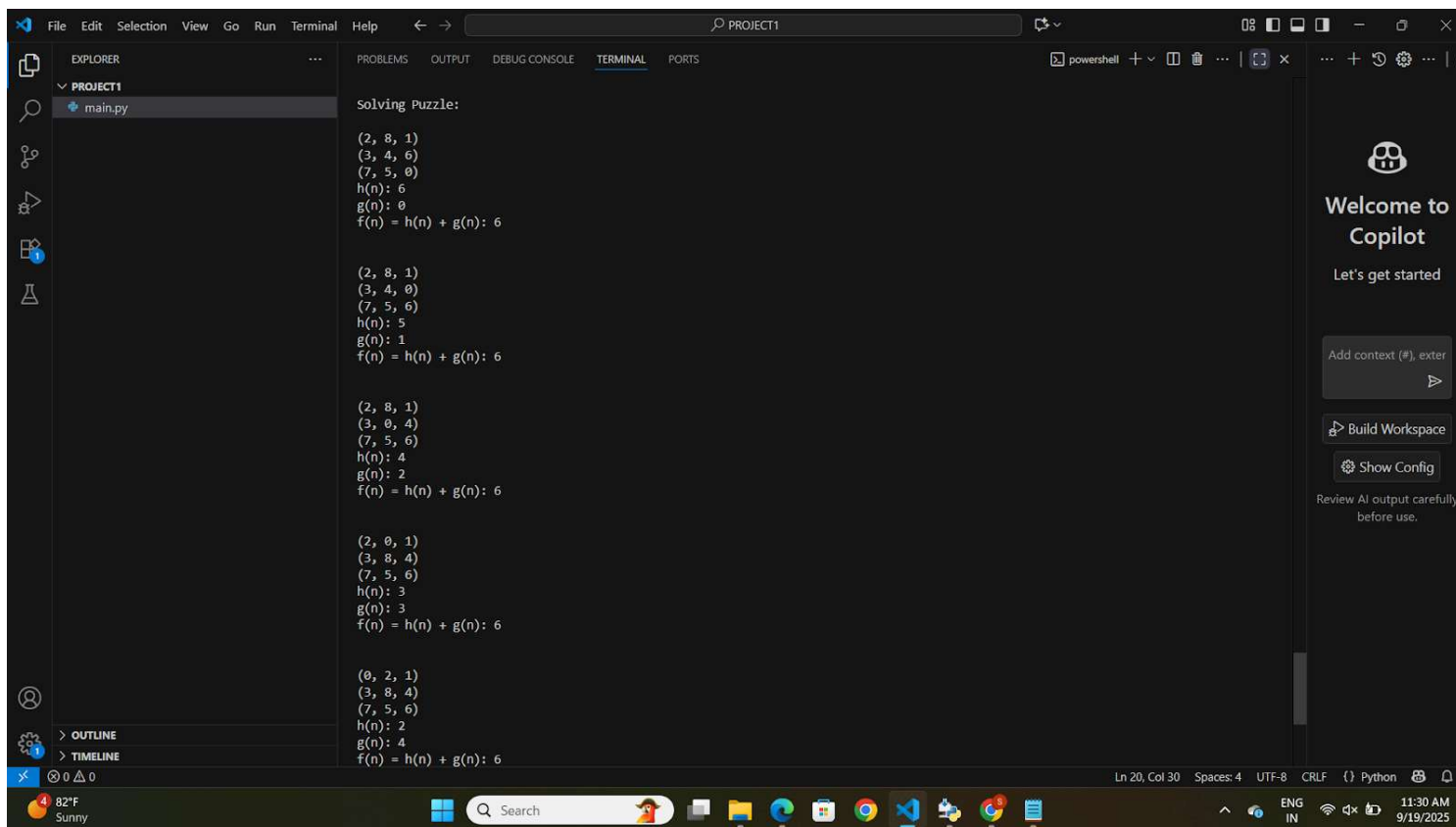
Initial State:

2	8	1
3	4	6
7	5	0

Goal State:

3	2	1
8	0	4
7	5	6

Manhattan Distance:



```
File Edit Selection View Go Run Terminal Help PROJECT1
Solving Puzzle:

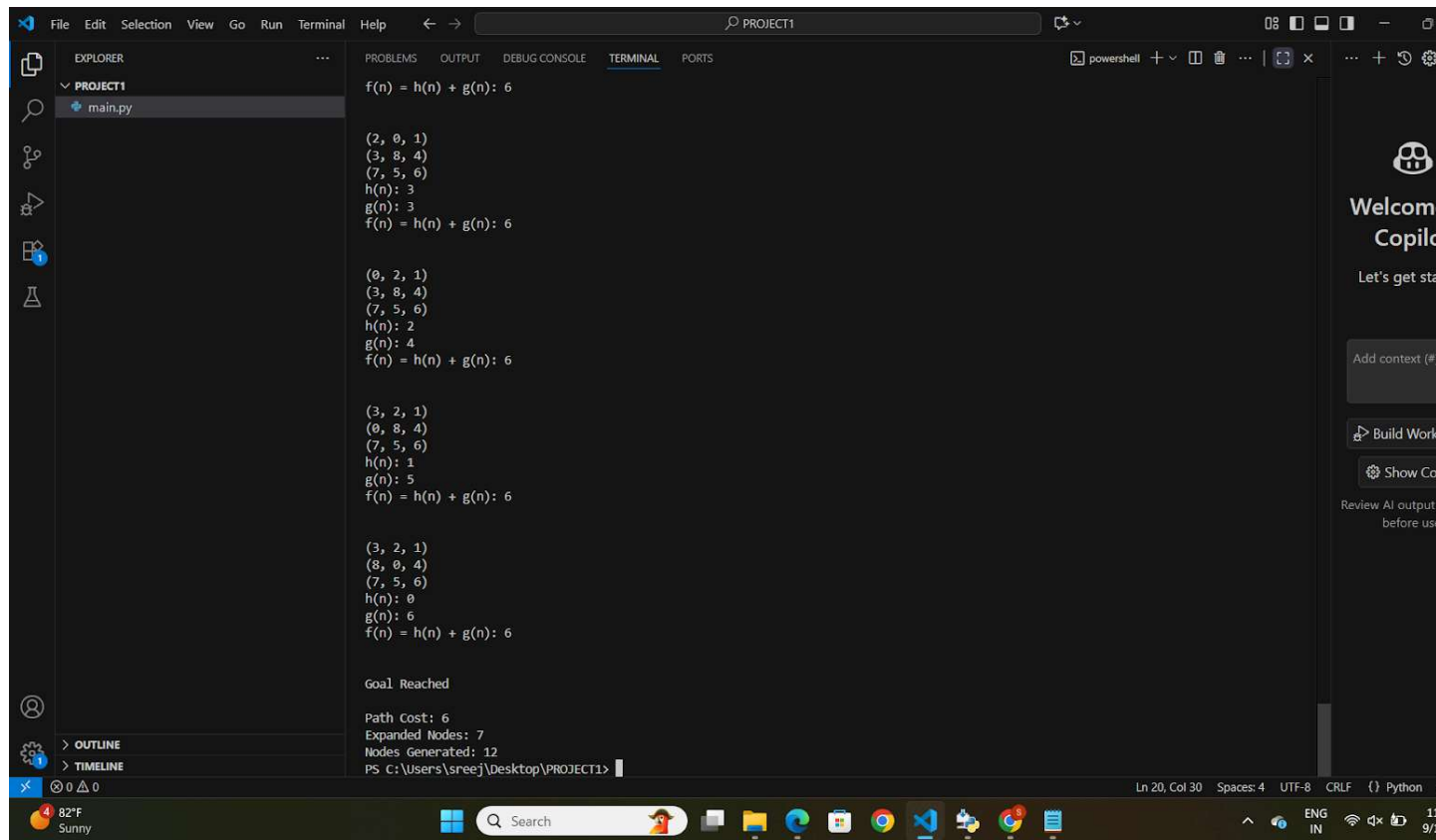
(2, 8, 1)
(3, 4, 6)
(7, 5, 0)
h(n): 6
g(n): 0
f(n) = h(n) + g(n): 6

(2, 8, 1)
(3, 4, 0)
(7, 5, 6)
h(n): 5
g(n): 1
f(n) = h(n) + g(n): 6

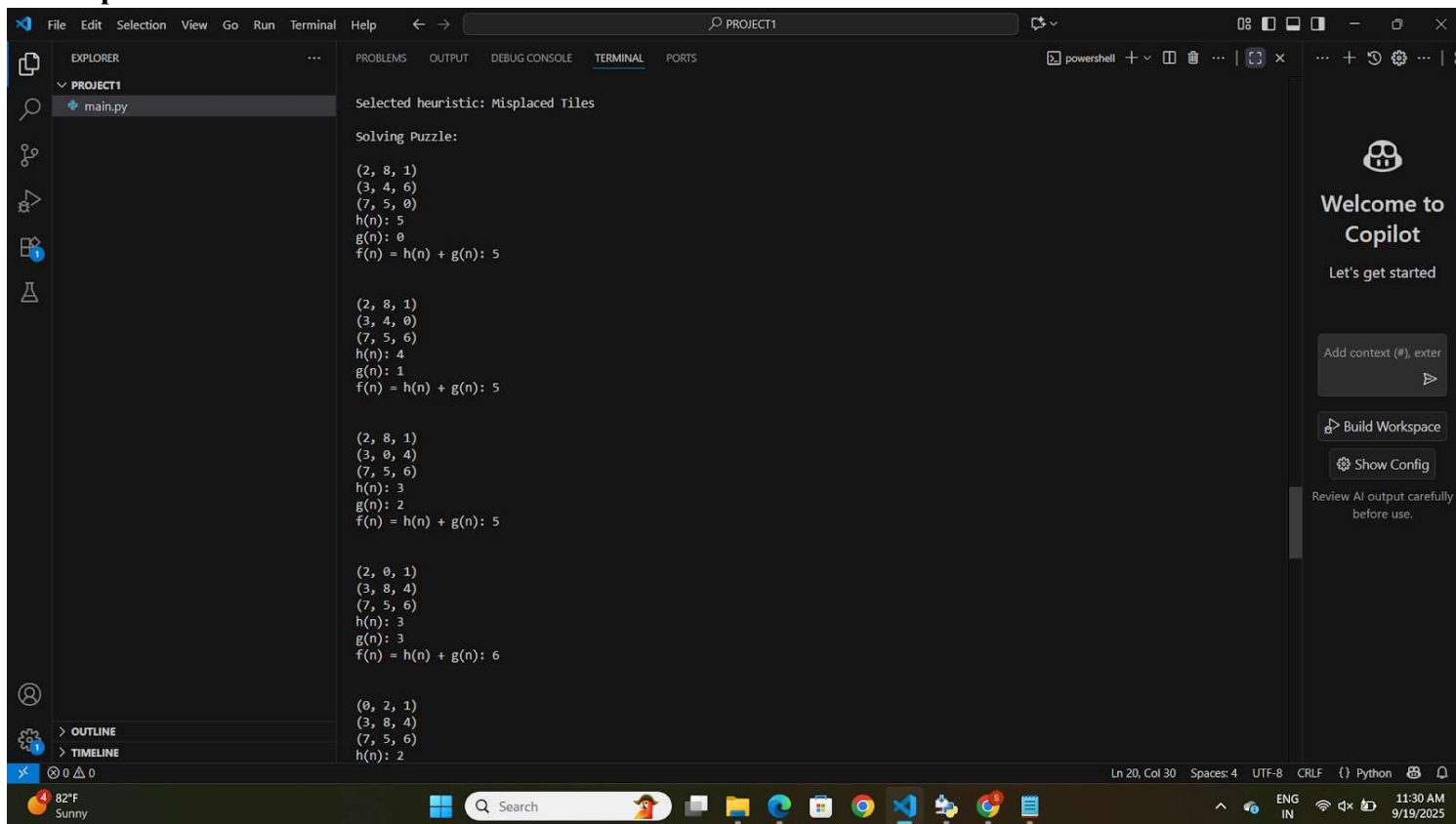
(2, 8, 1)
(3, 0, 4)
(7, 5, 6)
h(n): 4
g(n): 2
f(n) = h(n) + g(n): 6

(2, 0, 1)
(3, 8, 4)
(7, 5, 6)
h(n): 3
g(n): 3
f(n) = h(n) + g(n): 6

(0, 2, 1)
(3, 8, 4)
(7, 5, 6)
h(n): 2
g(n): 4
f(n) = h(n) + g(n): 6
```



Misplaced Tiles Distance:



FILE EDIT SELECTION VIEW GO RUN TERMINAL HELP

PROJECT1

powerShell

EXPLORER

PROJECT1

main.py

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(2, 0, 1)
(3, 8, 4)
(7, 5, 6)
h(n): 3
g(n): 3
f(n) = h(n) + g(n): 6

(0, 2, 1)
(3, 8, 4)
(7, 5, 6)
h(n): 2
g(n): 4
f(n) = h(n) + g(n): 6

(3, 2, 1)
(0, 8, 4)
(7, 5, 6)
h(n): 1
g(n): 5
f(n) = h(n) + g(n): 6

(3, 2, 1)
(8, 0, 4)
(7, 5, 6)
h(n): 0
g(n): 6
f(n) = h(n) + g(n): 6

Goal Reached

Path Cost: 6
Expanded Nodes: 8
Nodes Generated: 14
PS C:\Users\sreej\Desktop\PROJECT1> python main.py
Enter the initial puzzle state (9 numbers; 0 is blank).

WELCOME TO Copilot

Let's get started

ADD CONTEXT (#) ENTER

BUILD WORKSPACE

SHOW CONFIG

REVIEW AI OUTPUT CAREFULLY BEFORE USE.

OUTLINE

TIMELINE

0 0 0

82°F Sunny

SEARCH

11:30 AM 9/19/2025

Input 3:

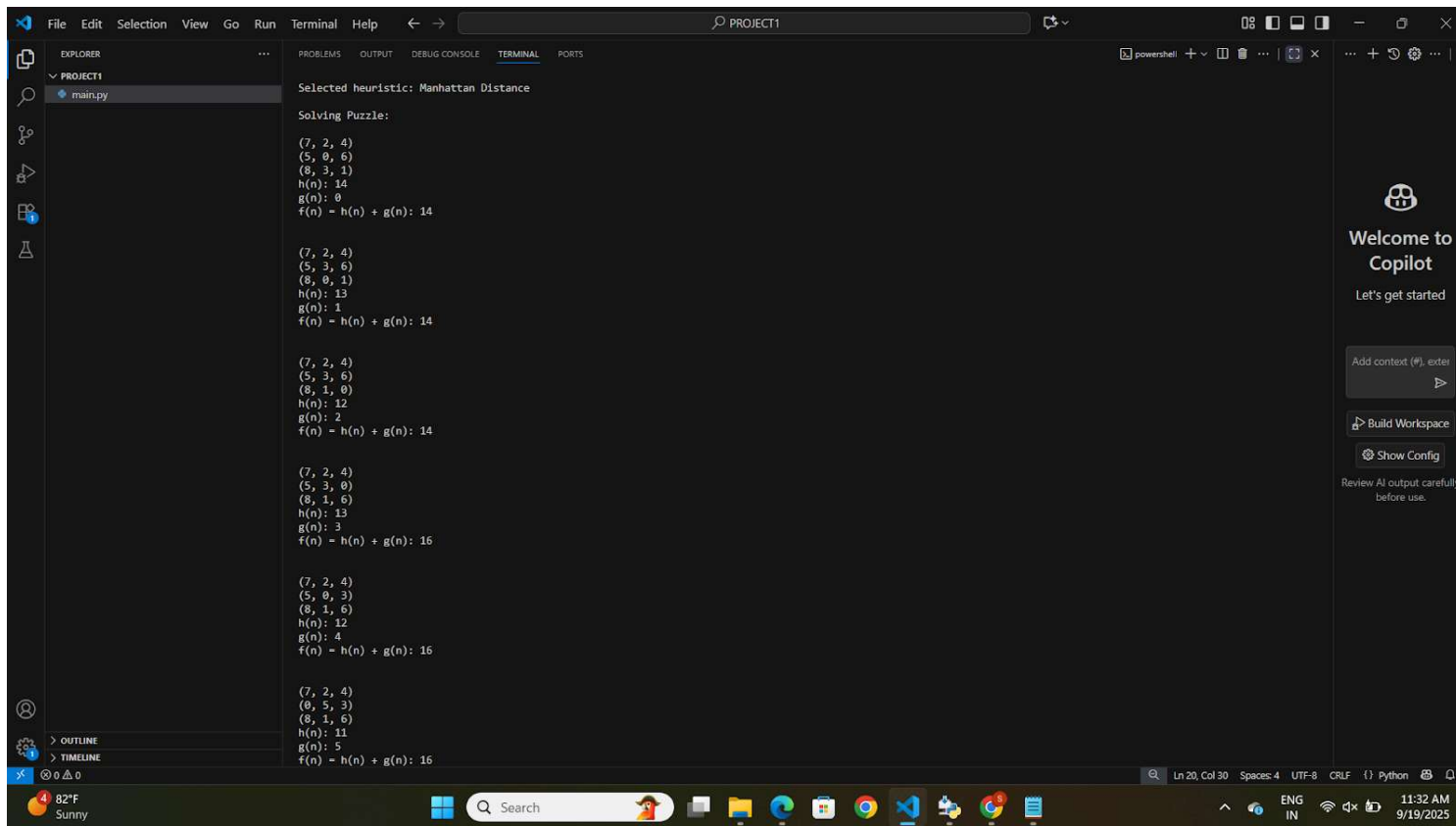
Initial State:

7	2	4
5	0	6
8	3	1

Goal State:

1	2	3
4	5	6
7	8	0

Manhattan Distance:



```
File Edit Selection View Go Run Terminal Help PROJECT1
EXPLORER PROJECT1 main.py
TERMINAL
Selected heuristic: Manhattan Distance
Solving Puzzle:
(7, 2, 4)
(5, 0, 6)
(8, 3, 1)
h(n): 14
g(n): 0
f(n) = h(n) + g(n): 14

(7, 2, 4)
(5, 3, 6)
(8, 0, 1)
h(n): 13
g(n): 1
f(n) = h(n) + g(n): 14

(7, 2, 4)
(5, 3, 0)
(8, 1, 6)
h(n): 12
g(n): 2
f(n) = h(n) + g(n): 14

(7, 2, 4)
(5, 3, 0)
(8, 1, 6)
h(n): 13
g(n): 3
f(n) = h(n) + g(n): 16

(7, 2, 4)
(8, 5, 3)
(8, 1, 6)
h(n): 12
g(n): 4
f(n) = h(n) + g(n): 16

(7, 2, 4)
(8, 5, 3)
(8, 1, 6)
h(n): 11
g(n): 5
f(n) = h(n) + g(n): 16
```

File Edit Selection View Go Run Terminal Help PROJECT1

EXPLORER PROJECT1 main.py

h(n): 11
g(n): 7
f(n) = h(n) + g(n): 18

(2, 4, 0)
(7, 5, 3)
(8, 1, 6)
h(n): 10
g(n): 8
f(n) = h(n) + g(n): 18

(2, 4, 3)
(7, 5, 0)
(8, 1, 6)
h(n): 9
g(n): 9
f(n) = h(n) + g(n): 18

(2, 4, 3)
(7, 0, 5)
(8, 1, 6)
h(n): 10
g(n): 10
f(n) = h(n) + g(n): 20

(2, 4, 3)
(7, 1, 5)
(8, 0, 6)
h(n): 9
g(n): 11
f(n) = h(n) + g(n): 20

(2, 4, 3)
(7, 1, 5)
(0, 8, 6)
h(n): 8
g(n): 12
f(n) = h(n) + g(n): 20

(2, 4, 3)
(0, 1, 5)
(7, 8, 6)
h(n): 7
g(n): 13

OUTLINE
TIMELINE

82°F Sunny

Search

Ln 20, Col 30 Spaces: 4 UTF-8 CRLF Python

Welcome to Copilot
Let's get started
Add context (#), enter
Build Workspace
Show Config
Review AI output carefully before use.

11:32 AM 9/19/2025

File Edit Selection View Go Run Terminal Help PROJECT1

EXPLORER PROJECT1 main.py

h(n): 5
g(n): 15
f(n) = h(n) + g(n): 20

(0, 2, 3)
(1, 4, 5)
(7, 8, 6)
h(n): 4
g(n): 16
f(n) = h(n) + g(n): 20

(1, 2, 3)
(0, 4, 5)
(7, 8, 6)
h(n): 3
g(n): 17
f(n) = h(n) + g(n): 20

(1, 2, 3)
(4, 0, 5)
(7, 8, 6)
h(n): 2
g(n): 18
f(n) = h(n) + g(n): 20

(1, 2, 3)
(4, 5, 0)
(7, 8, 6)
h(n): 1
g(n): 19
f(n) = h(n) + g(n): 20

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
h(n): 0
g(n): 20
f(n) = h(n) + g(n): 20

Goal Reached

Path Cost: 20
Expanded Nodes: 283
Nodes Generated: 451

OUTLINE
TIMELINE

82°F Sunny

Search

Ln 20, Col 30 Spaces: 4 UTF-8 CRLF Python

Welcome to Copilot
Let's get started
Add context (#), enter
Build Workspace
Show Config
Review AI output carefully before use.

11:33 AM 9/19/2025

Misplaced Tiles Distance:

The image shows a Visual Studio Code editor with a Python script and its terminal output. The script is titled "Misplaced Tiles Distance:" and is solving a puzzle. The terminal output shows the results of the search algorithm for various states.

Terminal Output:

```
Selected heuristic: Misplaced Tiles

Solving Puzzle:

(7, 2, 4)
(5, 0, 6)
(8, 3, 1)
h(n): 6
g(n): 0
f(n) = h(n) + g(n): 6

(7, 2, 4)
(5, 3, 6)
(8, 0, 1)
h(n): 6
g(n): 1
f(n) = h(n) + g(n): 7

(7, 2, 4)
(5, 3, 6)
(8, 1, 0)
h(n): 6
g(n): 2
f(n) = h(n) + g(n): 8

(7, 2, 4)
(5, 3, 0)
(8, 1, 6)
h(n): 7
g(n): 3
f(n) = h(n) + g(n): 10

(7, 2, 4)
(5, 0, 3)
(8, 1, 6)
h(n): 7
g(n): 4
f(n) = h(n) + g(n): 11

(7, 2, 4)
(0, 5, 3)
(8, 1, 6)
h(n): 6
g(n): 5

f(n) = h(n) + g(n): 14

(2, 4, 0)
(7, 5, 3)
(8, 1, 6)
h(n): 7
g(n): 8
f(n) = h(n) + g(n): 15

(2, 4, 3)
(7, 5, 0)
(8, 1, 6)
h(n): 6
g(n): 9
f(n) = h(n) + g(n): 15

(2, 4, 3)
(7, 0, 5)
(8, 1, 6)
h(n): 7
g(n): 10
f(n) = h(n) + g(n): 17

(2, 4, 3)
(7, 1, 5)
(8, 0, 6)
h(n): 7
g(n): 11
f(n) = h(n) + g(n): 18

(2, 4, 3)
(7, 1, 5)
(0, 8, 6)
h(n): 6
g(n): 12
f(n) = h(n) + g(n): 18

(2, 4, 3)
(0, 1, 5)
(7, 8, 6)
h(n): 5
g(n): 13
f(n) = h(n) + g(n): 18
```

The image shows the Visual Studio Code editor interface. The Explorer panel on the left shows a project named 'PROJECT1' with a file 'main.py'. The main editor area displays the code for 'main.py', which defines a recursive function $f(n) = h(n) + g(n): 20$. The terminal panel at the bottom shows the output of the program, which is a sequence of recursive calls and returns, culminating in 'Goal Reached'. The status bar at the bottom indicates the current line and column (Ln 20, Col 30) and the file encoding (UTF-8 CRLF Python). The right sidebar shows the 'Welcome to Copilot' message.

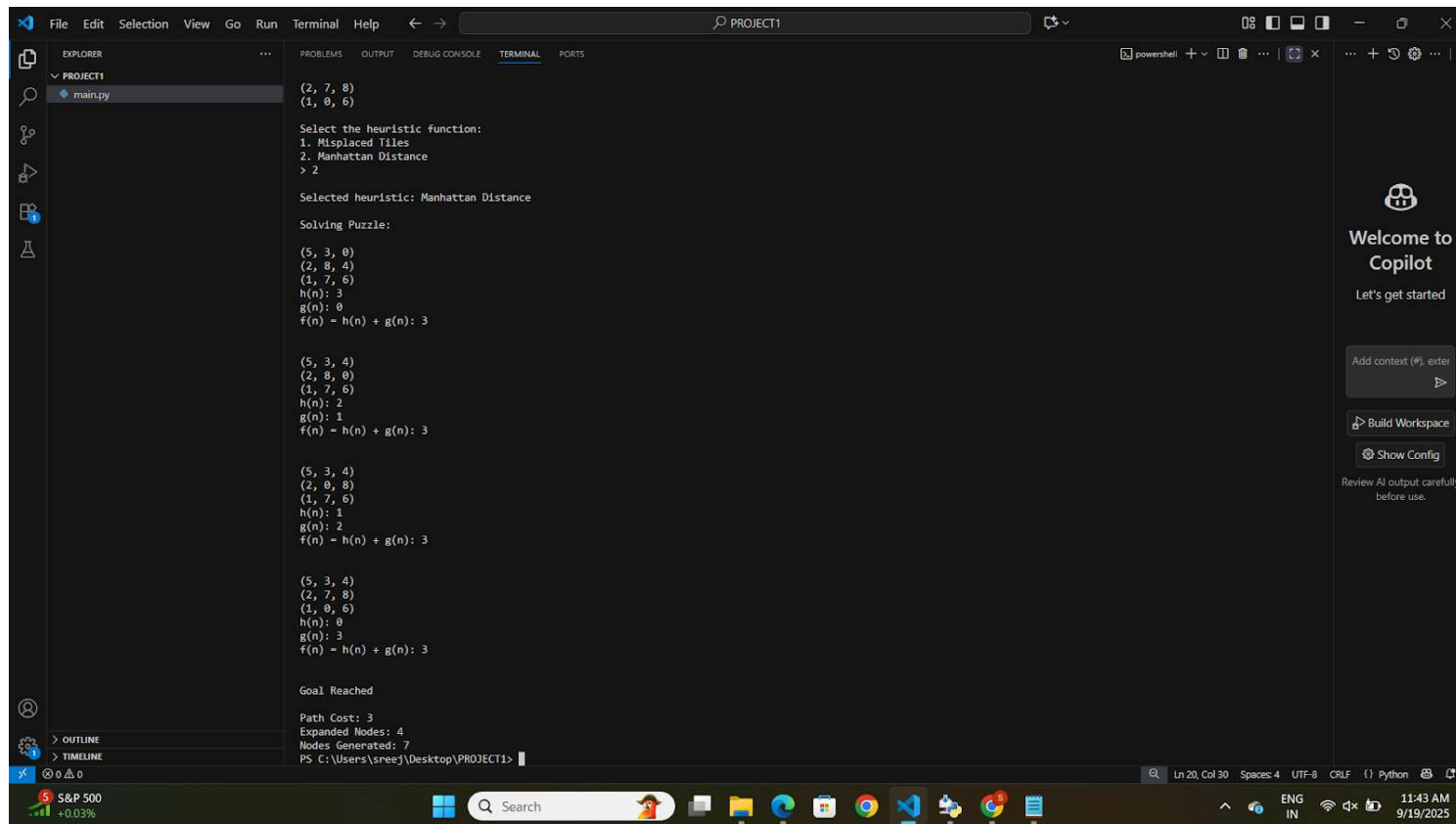
Input 4:

5	3	0
2	8	4
1	7	6

Goal State:

5	3	4
2	7	8
1	0	6

Manhattan Distance:



```
File Edit Selection View Go Run Terminal Help PROJECT1
EXPLORER
PROJECT1
main.py
TERMINAL
(2, 7, 8)
(1, 0, 6)

Select the heuristic function:
1. Misplaced Tiles
2. Manhattan Distance
> 2

Selected heuristic: Manhattan Distance

Solving Puzzle:

(5, 3, 0)
(2, 8, 4)
(1, 7, 6)
h(n): 3
g(n): 0
f(n) = h(n) + g(n): 3

(5, 3, 4)
(2, 8, 0)
(1, 7, 6)
h(n): 2
g(n): 1
f(n) = h(n) + g(n): 3

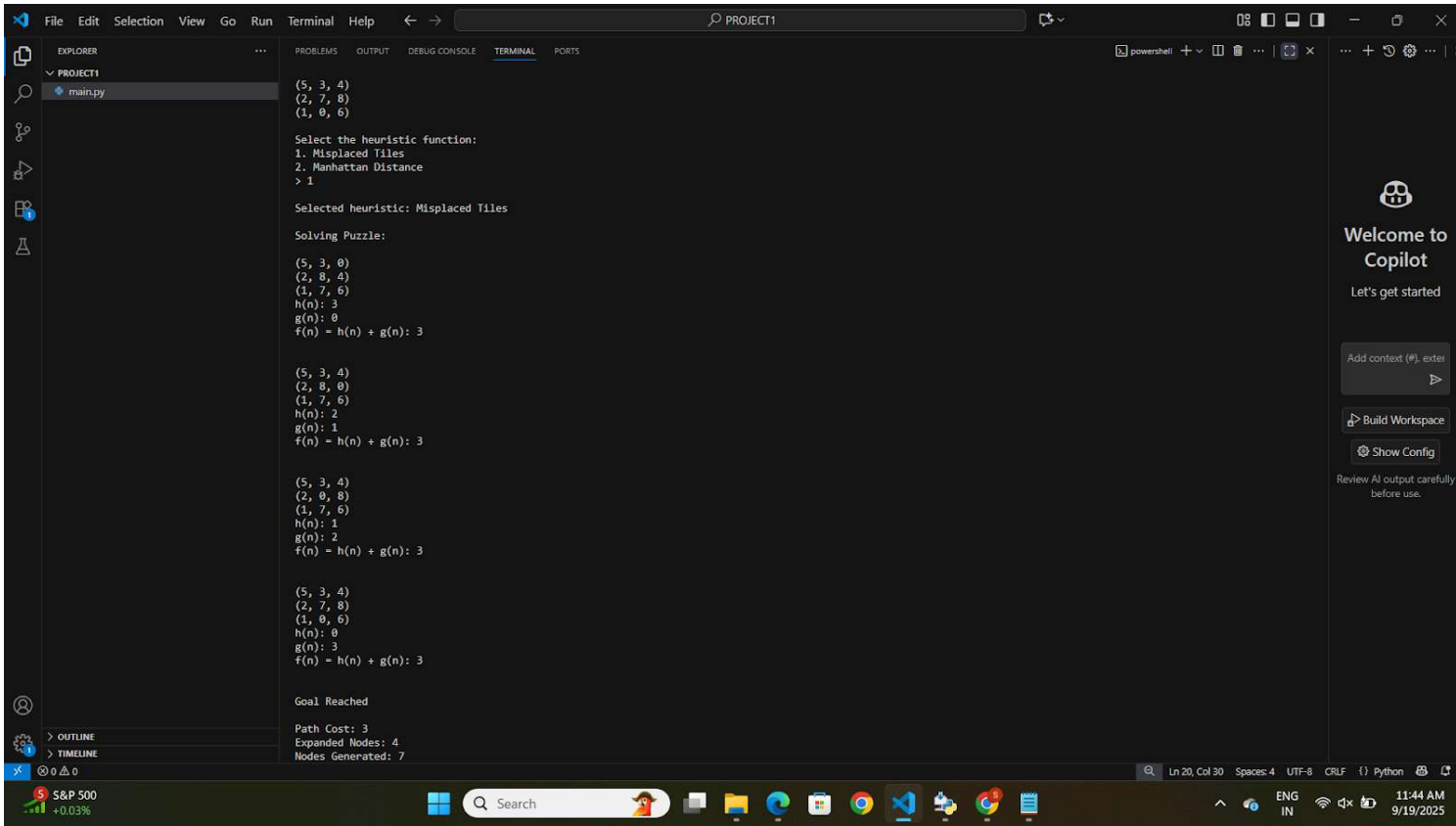
(5, 3, 4)
(2, 0, 8)
(1, 7, 6)
h(n): 1
g(n): 2
f(n) = h(n) + g(n): 3

(5, 3, 4)
(2, 7, 8)
(1, 0, 6)
h(n): 0
g(n): 3
f(n) = h(n) + g(n): 3

Goal Reached

Path Cost: 3
Expanded Nodes: 4
Nodes Generated: 7
PS C:\Users\sreej\Desktop\PROJECT1>
```

Misplaced Tiles Distance:



Input 5:

Initial State:

1	4	2
7	0	5
3	6	8

Goal State:

1	2	3
4	5	6
7	8	0

Manhattan Distance:

Solving Puzzle:

(1, 4, 2)

(7, 0, 5)

(3, 6, 8)

$h(n)$: 7

$g(n)$: 0

$f(n) = h(n) + g(n)$: 7

(1, 4, 2)

(7, 6, 5)

(3, 0, 8)

$h(n)$: 7

$g(n)$: 1

$f(n) = h(n) + g(n)$: 8

(1, 4, 2)

(7, 6, 5)

(0, 3, 8)

$h(n)$: 7

$g(n)$: 2

$f(n) = h(n) + g(n)$: 9

(1, 4, 2)

(0, 6, 5)

(7, 3, 8)

$h(n)$: 6

$g(n)$: 3

$f(n) = h(n) + g(n)$: 9

(1, 4, 2)

(6, 0, 5)

(7, 3, 8)

$h(n)$: 6

$g(n)$: 4

$f(n) = h(n) + g(n)$: 10

(1, 4, 2)

(6, 3, 5)

(7, 0, 8)

$h(n)$: 6

$g(n)$: 5

$f(n) = h(n) + g(n)$: 11

(1, 4, 2)
(6, 3, 5)
(7, 8, 0)
 $h(n)$: 5
 $g(n)$: 6
 $f(n) = h(n) + g(n)$: 11

(1, 4, 2)
(6, 3, 0)
(7, 8, 5)
 $h(n)$: 5
 $g(n)$: 7
 $f(n) = h(n) + g(n)$: 12

(1, 4, 2)
(6, 0, 3)
(7, 8, 5)
 $h(n)$: 5
 $g(n)$: 8
 $f(n) = h(n) + g(n)$: 13

(1, 4, 2)
(0, 6, 3)
(7, 8, 5)
 $h(n)$: 5
 $g(n)$: 9
 $f(n) = h(n) + g(n)$: 14

(1, 4, 2)
(7, 6, 3)
(0, 8, 5)
 $h(n)$: 6
 $g(n)$: 10
 $f(n) = h(n) + g(n)$: 16

(1, 4, 2)
(7, 6, 3)
(8, 0, 5)
 $h(n)$: 7
 $g(n)$: 11
 $f(n) = h(n) + g(n)$: 18

(1, 4, 2)
(7, 0, 3)
(8, 6, 5)
 $h(n)$: 7
 $g(n)$: 12
 $f(n) = h(n) + g(n)$: 19

(1, 0, 2)
(7, 4, 3)
(8, 6, 5)
 $h(n)$: 7
 $g(n)$: 13
 $f(n) = h(n) + g(n)$: 20

(1, 2, 0)
(7, 4, 3)
(8, 6, 5)
 $h(n)$: 6
 $g(n)$: 14
 $f(n) = h(n) + g(n)$: 20

(1, 2, 3)
(7, 4, 0)
(8, 6, 5)
 $h(n)$: 5
 $g(n)$: 15
 $f(n) = h(n) + g(n)$: 20

(1, 2, 3)
(7, 4, 5)
(8, 6, 0)
 $h(n)$: 5
 $g(n)$: 16
 $f(n) = h(n) + g(n)$: 21

(1, 2, 3)
(7, 4, 5)
(8, 0, 6)
 $h(n)$: 5
 $g(n)$: 17
 $f(n) = h(n) + g(n)$: 22

(1, 2, 3)
(7, 4, 5)
(0, 8, 6)
 $h(n)$: 4
 $g(n)$: 18
 $f(n) = h(n) + g(n)$: 22

(1, 2, 3)
(0, 4, 5)
(7, 8, 6)
 $h(n)$: 3
 $g(n)$: 19
 $f(n) = h(n) + g(n)$: 22

(1, 2, 3)
(4, 0, 5)
(7, 8, 6)
 $h(n)$: 2
 $g(n)$: 20
 $f(n) = h(n) + g(n)$: 22

(1, 2, 3)
(4, 5, 0)
(7, 8, 6)
 $h(n)$: 1
 $g(n)$: 21
 $f(n) = h(n) + g(n)$: 22

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
 $h(n)$: 0
 $g(n)$: 22
 $f(n) = h(n) + g(n)$: 22

Goal Reached

Path Cost: 22
Expanded Nodes: 10332
Nodes Generated: 15852

Misplaced Tiles Distance:

Solving Puzzle:

(1, 4, 2)
(7, 0, 5)
(3, 6, 8)
 $h(n)$: 7
 $g(n)$: 0
 $f(n) = h(n) + g(n)$: 7

(1, 4, 2)
(7, 6, 5)
(3, 0, 8)
 $h(n)$: 7
 $g(n)$: 1
 $f(n) = h(n) + g(n)$: 8

(1, 4, 2)
(7, 6, 5)
(0, 3, 8)
 $h(n)$: 7
 $g(n)$: 2
 $f(n) = h(n) + g(n)$: 9

(1, 4, 2)
(0, 6, 5)
(7, 3, 8)
 $h(n)$: 6
 $g(n)$: 3
 $f(n) = h(n) + g(n)$: 9

(1, 4, 2)
(6, 0, 5)
(7, 3, 8)
 $h(n)$: 6
 $g(n)$: 4
 $f(n) = h(n) + g(n)$: 10

(1, 4, 2)
(6, 3, 5)
(7, 0, 8)
 $h(n)$: 6
 $g(n)$: 5
 $f(n) = h(n) + g(n)$: 11

(1, 4, 2)
(6, 3, 5)
(7, 8, 0)

$h(n)$: 5
 $g(n)$: 6
 $f(n) = h(n) + g(n)$: 11

(1, 4, 2)
(6, 3, 0)
(7, 8, 5)
 $h(n)$: 5
 $g(n)$: 7
 $f(n) = h(n) + g(n)$: 12

(1, 4, 2)
(6, 0, 3)
(7, 8, 5)
 $h(n)$: 5
 $g(n)$: 8
 $f(n) = h(n) + g(n)$: 13

(1, 4, 2)
(0, 6, 3)
(7, 8, 5)
 $h(n)$: 5
 $g(n)$: 9
 $f(n) = h(n) + g(n)$: 14

(1, 4, 2)
(7, 6, 3)
(0, 8, 5)
 $h(n)$: 6
 $g(n)$: 10
 $f(n) = h(n) + g(n)$: 16

(1, 4, 2)
(7, 6, 3)
(8, 0, 5)
 $h(n)$: 7
 $g(n)$: 11
 $f(n) = h(n) + g(n)$: 18

(1, 4, 2)
(7, 0, 3)
(8, 6, 5)
 $h(n)$: 7
 $g(n)$: 12
 $f(n) = h(n) + g(n)$: 19

(1, 0, 2)
(7, 4, 3)
(8, 6, 5)
 $h(n)$: 7
 $g(n)$: 13
 $f(n) = h(n) + g(n)$: 20

(1, 2, 0)
(7, 4, 3)
(8, 6, 5)
h(n): 6
g(n): 14
f(n) = h(n) + g(n): 20

(1, 2, 3)
(7, 4, 0)
(8, 6, 5)
h(n): 5
g(n): 15
f(n) = h(n) + g(n): 20

(1, 2, 3)
(7, 4, 5)
(8, 6, 0)
h(n): 5
g(n): 16
f(n) = h(n) + g(n): 21

(1, 2, 3)
(7, 4, 5)
(8, 0, 6)
h(n): 5
g(n): 17
f(n) = h(n) + g(n): 22

(1, 2, 3)
(7, 4, 5)
(0, 8, 6)
h(n): 4
g(n): 18
f(n) = h(n) + g(n): 22

(1, 2, 3)
(0, 4, 5)
(7, 8, 6)
h(n): 3
g(n): 19
f(n) = h(n) + g(n): 22

(1, 2, 3)
(4, 0, 5)
(7, 8, 6)
h(n): 2
g(n): 20
f(n) = h(n) + g(n): 22

(1, 2, 3)

(4, 5, 0)
(7, 8, 6)
 $h(n)$: 1
 $g(n)$: 21
 $f(n) = h(n) + g(n)$: 22

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
 $h(n)$: 0
 $g(n)$: 22
 $f(n) = h(n) + g(n)$: 22

Goal Reached

Path Cost: 22
Expanded Nodes: 10332
Nodes Generated: 15852

Input 6:

Initial State:

5	1	3
4	2	6
7	8	0

Goal State:

5	1	3
4	2	0
7	8	6

Manhattan Distance:

Solving Puzzle:

(5, 1, 3)

(4, 2, 6)

(7, 8, 0)

$h(n)$: 1

$g(n)$: 0

$f(n) = h(n) + g(n)$: 1

(5, 1, 3)

(4, 2, 0)

(7, 8, 6)

$h(n)$: 0

$g(n)$: 1

$f(n) = h(n) + g(n)$: 1

Goal Reached

Path Cost: 1

Expanded Nodes: 2

Nodes Generated: 2

Misplaced Tiles Distance:

Solving Puzzle:

(5, 1, 3)

(4, 2, 6)

(7, 8, 0)

$h(n)$: 1

$g(n)$: 0

$f(n) = h(n) + g(n)$: 1

(5, 1, 3)

(4, 2, 0)

(7, 8, 6)

$h(n)$: 0

$g(n)$: 1

$f(n) = h(n) + g(n)$: 1

Goal Reached

Path Cost: 1

Expanded Nodes: 2

Nodes Generated: 2

Input 7:

Initial State:

Initial State:		
2	8	3
1	0	4
7	6	5
Goal State:		
1	2	3
8	0	4
7	6	5

Manhattan Distance:

Solving Puzzle:

(2, 8, 3)

(1, 0, 4)

(7, 6, 5)

h(n): 4

g(n): 0

f(n) = h(n) + g(n): 4

(2, 0, 3)

(1, 8, 4)

(7, 6, 5)

h(n): 3

g(n): 1

f(n) = h(n) + g(n): 4

(0, 2, 3)

(1, 8, 4)

(7, 6, 5)

h(n): 2

g(n): 2

f(n) = h(n) + g(n): 4

(1, 2, 3)
(0, 8, 4)
(7, 6, 5)
h(n): 1
g(n): 3
f(n) = h(n) + g(n): 4

(1, 2, 3)
(8, 0, 4)
(7, 6, 5)
h(n): 0
g(n): 4
f(n) = h(n) + g(n): 4

Goal Reached

Path Cost: 4
Expanded Nodes: 5

Nodes Generated: 9

Misplaced Tiles Distance:

Solving Puzzle:

(2, 8, 3)

(1, 0, 4)

(7, 6, 5)

$h(n)$: 3

$g(n)$: 0

$f(n) = h(n) + g(n)$: 3

(2, 0, 3)

(1, 8, 4)

(7, 6, 5)

$h(n)$: 3

$g(n)$: 1

$f(n) = h(n) + g(n)$: 4

(0, 2, 3)

(1, 8, 4)

(7, 6, 5)

$h(n)$: 2

$g(n)$: 2

$f(n) = h(n) + g(n)$: 4

(1, 2, 3)

(0, 8, 4)

(7, 6, 5)

$h(n)$: 1

$g(n)$: 3

$f(n) = h(n) + g(n)$: 4

(1, 2, 3)

(8, 0, 4)

(7, 6, 5)

$h(n)$: 0

$g(n)$: 4

$f(n) = h(n) + g(n)$: 4

Goal Reached

Path Cost: 4

Expanded Nodes: 6

Nodes Generated: 11

Input 8:

Initial State:

1	2	3
0	5	6
6	7	8

Goal State:

1	2	3
4	5	0
6	7	8

Manhattan Distance:

Solving Puzzle:

(1, 2, 3)

(0, 5, 6)

(4, 7, 8)

$h(n)$: 4

$g(n)$: 0

$f(n) = h(n) + g(n)$: 4

(1, 2, 3)

(5, 0, 6)

(4, 7, 8)

$h(n)$: 5

$g(n)$: 1

$f(n) = h(n) + g(n)$: 6

(1, 2, 3)

(5, 6, 0)

(4, 7, 8)

$h(n)$: 4

$g(n)$: 2

$$f(n) = h(n) + g(n): 6$$

(1, 2, 3)

(5, 6, 8)

(4, 7, 0)

$h(n): 5$

$g(n): 3$

$$f(n) = h(n) + g(n): 8$$

(1, 2, 3)

(5, 6, 8)

(4, 0, 7)

$h(n): 6$

$g(n): 4$

$$f(n) = h(n) + g(n): 10$$

(1, 2, 3)

(5, 0, 8)

(4, 6, 7)

$h(n): 5$

$g(n): 5$

$$f(n) = h(n) + g(n): 10$$

(1, 2, 3)

(0, 5, 8)

(4, 6, 7)

$h(n): 4$

$g(n): 6$

$$f(n) = h(n) + g(n): 10$$

(1, 2, 3)

(4, 5, 8)

(0, 6, 7)

$h(n): 3$

$g(n): 7$

$$f(n) = h(n) + g(n): 10$$

(1, 2, 3)

(4, 5, 8)

(6, 0, 7)

$h(n): 2$

$g(n): 8$

$$f(n) = h(n) + g(n): 10$$

(1, 2, 3)
(4, 5, 8)
(6, 7, 0)
h(n): 1
g(n): 9
f(n) = h(n) + g(n): 10

(1, 2, 3)
(4, 5, 0)
(6, 7, 8)
h(n): 0
g(n): 10
f(n) = h(n) + g(n): 10

Goal Reached

Path Cost: 10
Expanded Nodes: 37
Nodes Generated: 63

Selected heuristic: Misplaced Tiles

Solving Puzzle:

(1, 2, 3)
(0, 5, 6)
(4, 7, 8)
h(n): 2
g(n): 0
f(n) = h(n) + g(n): 2

(1, 2, 3)
(5, 0, 6)
(4, 7, 8)
h(n): 3
g(n): 1
f(n) = h(n) + g(n): 4

(1, 2, 3)
(5, 6, 0)
(4, 7, 8)
h(n): 3
g(n): 2
f(n) = h(n) + g(n): 5

(1, 2, 3)

(5, 6, 8)
(4, 7, 0)
 $h(n)$: 4
 $g(n)$: 3
 $f(n) = h(n) + g(n)$: 7

(1, 2, 3)
(5, 6, 8)
(4, 0, 7)
 $h(n)$: 5
 $g(n)$: 4
 $f(n) = h(n) + g(n)$: 9

(1, 2, 3)
(5, 0, 8)
(4, 6, 7)
 $h(n)$: 5
 $g(n)$: 5
 $f(n) = h(n) + g(n)$: 10

(1, 2, 3)
(0, 5, 8)
(4, 6, 7)
 $h(n)$: 4
 $g(n)$: 6
 $f(n) = h(n) + g(n)$: 10

(1, 2, 3)
(4, 5, 8)
(0, 6, 7)
 $h(n)$: 3
 $g(n)$: 7
 $f(n) = h(n) + g(n)$: 10

(1, 2, 3)
(4, 5, 8)
(6, 0, 7)
 $h(n)$: 2
 $g(n)$: 8
 $f(n) = h(n) + g(n)$: 10

(1, 2, 3)
(4, 5, 8)
(6, 7, 0)
 $h(n)$: 1
 $g(n)$: 9
 $f(n) = h(n) + g(n)$: 10

(1, 2, 3)
(4, 5, 0)
(6, 7, 8)
 $h(n)$: 0
 $g(n)$: 10

$f(n) = h(n) + g(n): 10$

Goal Reached

Path Cost: 10

Expanded Nodes: 69

Nodes Generated: 115

CONCLUSION

Therefore, by combining two different heuristic functions—Misplaced Tiles and Manhattan Distance—the A* algorithm has been successfully implemented in Python. The final output of this implementation is guaranteed to be the optimal path.